同济大学计算机系

操作系统课程实验报告

学　　号 2353761

姓　　名 王皙晶

专　　业 计算机科学与技术

授课老师 方钰

实验二：UNIX V6++ 进程的栈帧

一、实验目的

本次实验将使用 QEMU 模拟器来研究 IA-32 兼容计算机的启动方式。

（1）编写一个简单的 C++代码，并在 UNIX V6++中编译和运行调试，观察程序运行时栈帧的变化；

（2）进一步掌握 UNIX V6++重新编译及运行调试的方法。

二、实验结果

（1）4.1~4.3，掌握在 UNIX V6++中添加自定义程序及编译、链接与运行的全过程，掌握 UNIX V6++中调试运行与观察结果的常规操作。通过观察内存单元的值验证用户栈的变化：

4.1 在 UNIX V6++中编译链接运行一个 C 语言程序：

实验结果如图 1 所示：

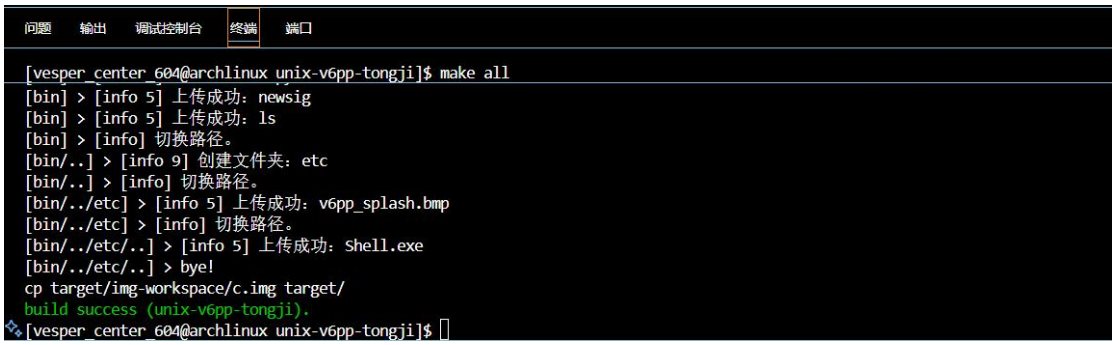

图 1

4.2. 开始程序的调试运行：

实验结果如图 2 所示：

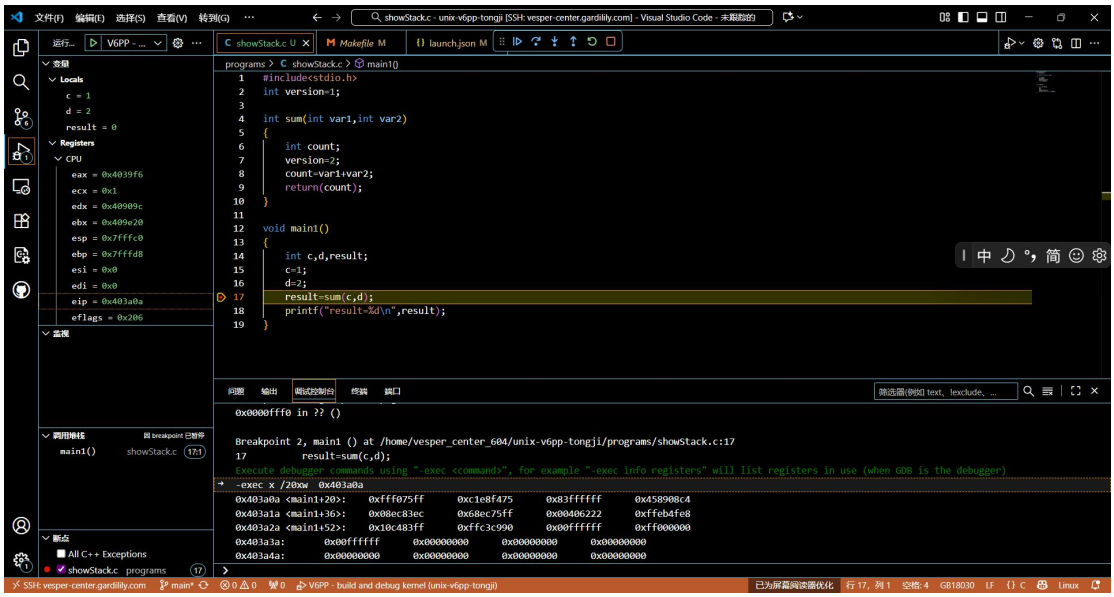

图 2

4.3. 观察 MAIN1 堆栈变化（通过汇编和内存单元值的变化）：

实验结果如图 3 所示：

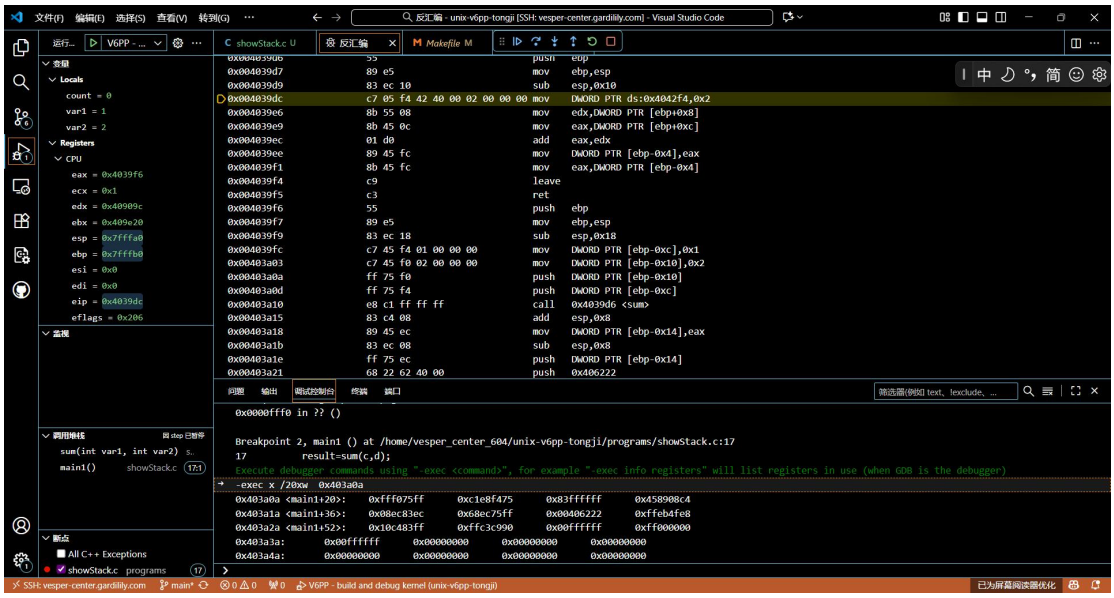

图 3

（2）完成实验 4.4，通过观察内存单元的值，参考图 14-15 绘制图表，详细说明在 sum 执行的过程中，用户栈的变化。结合你对 sum 函数的分析，回答 4.4 中的两个问题：

Sum 函数汇编指令注解如下：

```
==============================================================
push ebp                          //main函数栈帧的ebp存入当前栈
mov ebp, esp                      //修改ebp指向当前栈帧，此时ebp=0x7fffb0
sub esp, 0x10                     //esp上移4个字
mov DWORD PTR ds:0x4042f4, 0x2    //修改在内存0x4042f4的全局变量version值为2
mov edx, DWORD PTR [ebp+0x8]      //将存在ebp+0x8的参数1给edx，为add做准备
mov eax, DWORD PTR [ebp+0xc]      //将存在ebp+0xc的参数2给eax，为add做准备
add eax, edx                      //加法运算
mov DWORD PTR [ebp-0x4], eax      //将加法运算的值赋给局部变量count
mov eax, DWORD PTR [ebp-0x4]
leave
ret                               //结束调用，恢复ebp和esp到调用前的值
==============================================================
```

结合上述本人对汇编指令的注解，sum 函数执行过程中用户栈的变化如下：

首先将 call 函数的下一句地址压入栈，如图 4 所示；再将 main 函数 ebp 的值压入栈，如图 5 所示；再将执行完加法运算的局部变量 count 的值压入栈中，如图 6 所示；

	0x7fffb4	0x00403a15	函数调用语句call的下一句地址
	0x007fffb8	1	push DWORD PTR [ebp-0xc]
	0x007fffbc	2	push DWORD PTR [ebp-0x10]
main 栈帧	0x007fffc0	0x00000000	此时 ESP 的值
	0x007fffc4	0x00000000	
	0x007fffc8	2	mov DWORD PTR [ebp-0x10], 0x2
	0x007fffcc	1	mov DWORD PTR [ebp-0xc], 0x1
	0x007fffd0	0x00000000	
	0x007fffd4	0x00000000	
	0x007fffd8	0x007FFFE0	此时 EBP 的值（其中的值为前一栈帧的 EBP）
高地址	0x007fffdc	0x00000008	main 的返回地址

图 4

	0x7fffb0	0x007fffd8	此时EBP的值，为main函数栈帧的EBP
	0x7fffb4	0x00403a15	函数调用语句call的下一句地址
	0x007fffb8	1	push DWORD PTR [ebp-0xc]
	0x007fffbc	2	push DWORD PTR [ebp-0x10]
main 栈帧	0x007fffc0	0x00000000	此时 ESP 的值
	0x007fffc4	0x00000000	
	0x007fffc8	2	mov DWORD PTR [ebp-0x10], 0x2
	0x007fffcc	1	mov DWORD PTR [ebp-0xc], 0x1
	0x007fffd0	0x00000000	
	0x007fffd4	0x00000000	
	0x007fffd8	0x007FFFE0	此时 EBP 的值（其中的值为前一栈帧的 EBP）
高地址	0x007fffdc	0x00000008	main 的返回地址

图 5

	0x7fffac	3	mov DWORD PTR[ebp-0x4],eax
	0x7fffb0	0x007fffd8	此时EBP的值，为main函数栈帧的EBP
	0x7fffb4	0x00403a15	函数调用语句call的下一句地址
	0x007fffb8	1	push DWORD PTR [ebp-0xc]
	0x007fffbc	2	push DWORD PTR [ebp-0x10]
main 栈帧	0x007fffc0	0x00000000	此时 ESP 的值
	0x007fffc4	0x00000000	
	0x007fffc8	2	mov DWORD PTR [ebp-0x10], 0x2
	0x007fffcc	1	mov DWORD PTR [ebp-0xc], 0x1
	0x007fffd0	0x00000000	
	0x007fffd4	0x00000000	
	0x007fffd8	0x007FFFE0	此时 EBP 的值（其中的值为前一栈帧的 EBP）
高地址	0x007fffdc	0x00000008	main 的返回地址

图 6

（1）在 main1 的汇编代码中，从 sum 返回后执行的指令“add esp, 0x8”的目的是什么？

目的是彻底清理 sum 函数的栈帧。从 sum 函数返回后 esp 仍指向上图所示的 0x7fffb4，此时还保留调用 sum 函数压入的参数值。将 esp 的值加 0x8 后则指向 0x7fffc0，相当于清理了上述调用值。

（2）在 sum 的汇编代码中，“mov DWORD PTR ds:0x4042f4, 0x2”的作用是什么？其中 ds:0x4042f4 这个地址中保存的是什么？能否用调试过程来证明你的答案？

该指令的作用是给存在 0x4042f4 的全局变量 version 赋值为 2。如图 7 所示，执行这一指令前，该内存处存的值是 1，刚好对应一开始初始化时 version 的值；执行这指令后，如图 8 所示，内存处的值就改为 2 了。

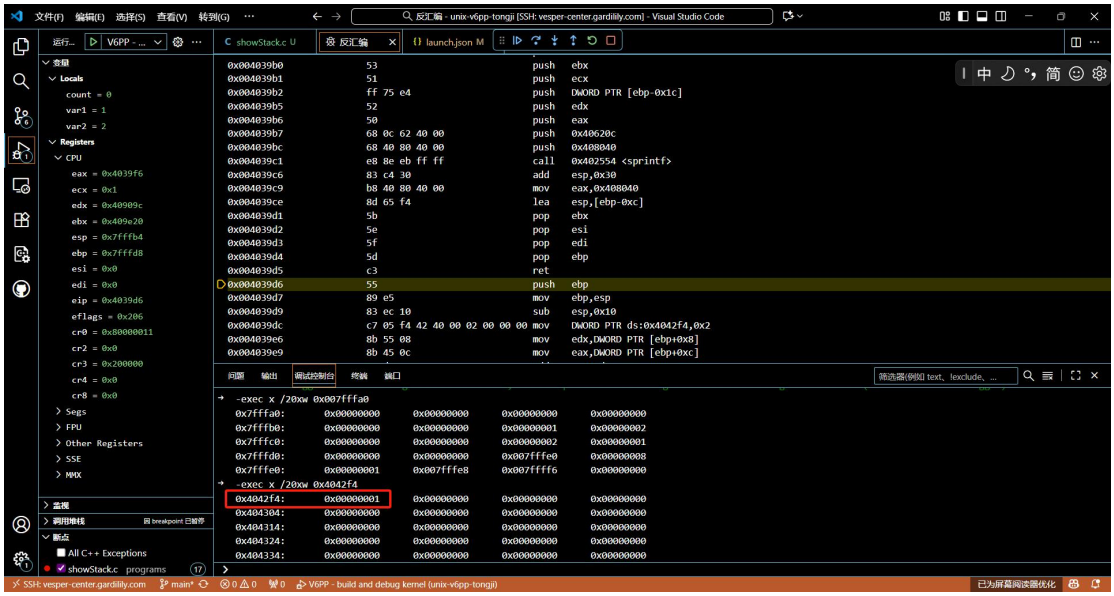

图 7

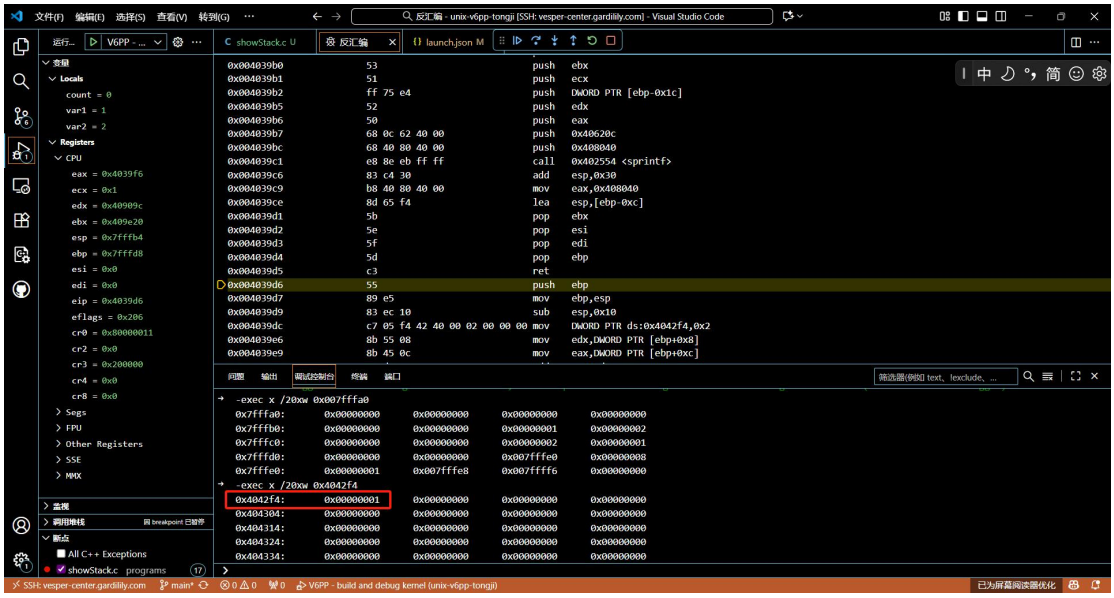

图 8

（3）完成实验 4.5，结合调试过程，解释程序的输出，并按要求修改代码：

未修改前，程序输出如图 9 所示。

图 9

根据执行赋值语句后，内存单元的值（如图 10 所示）可以推断出，之所以输出有误是由于按照给定程序那样赋值，没有留出尾零的位置，所以按照字符串方式输出时，会一直从起始地址输出指导遇到尾零（0x00）。所以按照给定程序那样从图 10 所示的内存处输出，先从 0x7fffc9 处的 0x48（'H'）开始输出，直到 0x7fff6f 处的'o',后面遇到初始化时的 0x00，认为遇到了尾 0，于是结束串方式输出。但是在输出"World!"时，最后一个'！'（0x7fffca 处的 0x21）后面并没有尾 0 结束串方式输出，所以就顺着输出刚好存储在其之后的"Hello"了，又遇到尾 0，结束输出。

```
-exec x /20xw 0x007fffc0
0x7fffc0:       0x00000000      0x726f5700      0x4821646c      0x6f6c6c65
0x7fffd0:       0x00000000      0x00000000      0x007fffe0      0x00000008
0x7fffe0:       0x00000001      0x007fffe8      0x007ffff6      0x00000000
0x7ffff0:       0x00000000      0x68730000      0x7453776f      0x006b6361
0x800000:       Cannot access memory at address 0x800000
```

图 10

基于以上分析，很容易知道程序该如何修改。当然最常见的就是在字符串初始化的时候，给定字符数组的长度为字符个数+1，预留出尾 0 的位置。在当前内存管理下，也可以只给 b 字符串字符数组的长度+1，留一个给尾 0，即可正常输出。程序修改结果见图 11，内存单元结果见图 12，正常输出结果见图 13。

showStack.c U × 反汇编 launch.json M

programs > showStack.c > main1()

```c
int sum(int var1,int var2)
    version=2;
    count=var1+var2;
    return(count);
}

void main1()
{
    //int c,d,result;
    //c=1;
    //d=2;
    //result=sum(c,d);
    //printf("result=%d\n",result);

    char a[5]="Hello";
    char b[7]="World!";
    printf("%s %s\n",a,b);
}
```

图 11

```
-exec x /20xw 0x007fffc0
0x7fffc0:       0x00000000      0x6c726f57      0x48002164      0x6f6c6c65
0x7fffd0:       0x00000000      0x00000000      0x007fffe0      0x00000008
0x7fffe0:       0x00000001      0x007fffe8      0x007ffff6      0x00000000
0x7ffff0:       0x00000000      0x68730000      0x7453776f      0x006b6361
0x800000:       Cannot access memory at address 0x800000
```

图 12

QEMU ×

Machine View

```
Welcome to Unix V6++ Tongji's Edition!
[/]#cd bin
[/bin]#showStack
Hello World!
[/bin]#_

Process 2 (Status:5)  end wait
```

图 13